

神经网络算法

神经网络算法是一种监督学习算法，可用于解决分类问题或回归问题，最大特色是对模型结构和假设施加最小需求，可以接近各种统计模型，并不需要先假设响应变量和特征变量间的特定关系，响应变量和特征变量之间的特定关系在学习过程中确定。这一优势使其应用非常广泛，成为当前最为流行的机器学习算法之一，比如将神经网络算法应用到商业银行授信客户的信用风险评估中，给授信客户评分以获取其拟违约的概率，从而判断新授信或续授信业务风险；将神经网络算法应用到房地产客户电话营销中，用于预测对电话营销作出响应的概率，从而更加合理的配置营销资源；将神经网络算法应用到制造业中，用于预测目标客户群体的购买需求，从而可以制定针对性的生产策略，以合理控制成本。本章我们讲解神经网络算法的基本原理，并结合具体实例讲解该算法在python中解决分类问题和回归问题的实现与应用。

目录

C O N T E N T S

- 1 神经网络算法的基本原理
- 2 数据准备
- 3 回归神经网络算法示例
- 4 二分类神经网络算法示例
- 5 多分类神经网络算法示例
- 6 习 题



PART 01

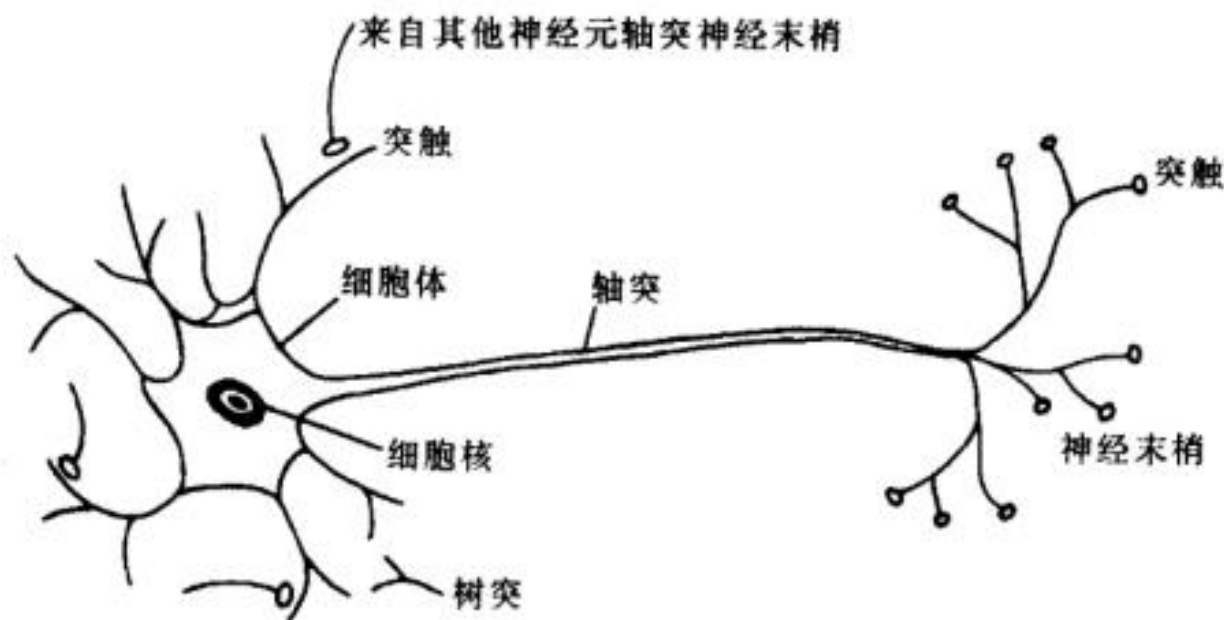
神经网络算法的基本原理

神经网络算法的基本思想

神经网络算法的基本思想是对人脑神经网络进行抽象,通过模拟人脑神经网络的连接机制来建立起算法模型,实现机器学习。

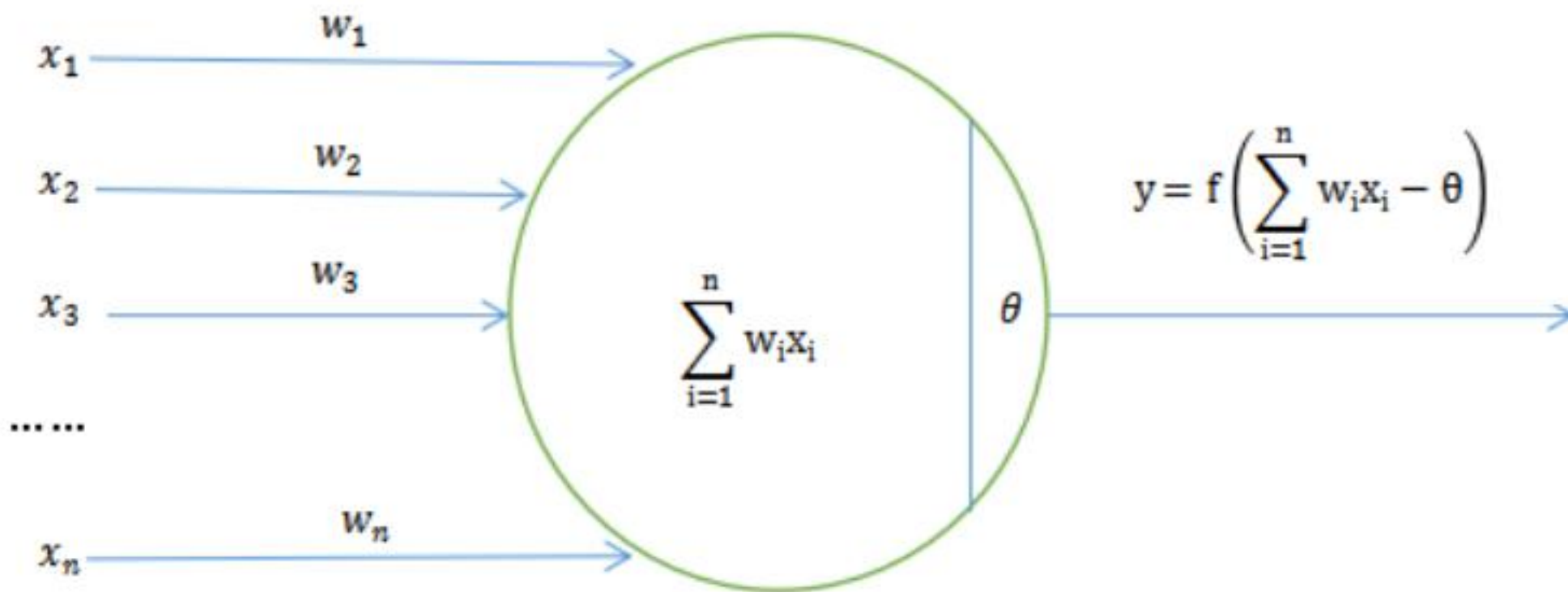
在人脑神经网络中,最为基本的单位是神经元(也称神经细胞),如下图所示(图片来自于互联网),神经元左侧有很多树突,树突接受来自其他神经元的信号,树突的分支上有树突小芽,与其他神经元的神经末梢形成突触。

一个典型的神经元通过左侧的众多树突,从其他神经元处突触获得信号,然后细胞体针对获得的一系列信号进行加权计算,如果计算后认为超过了兴奋阈值,那么该神经元就会兴奋起来,并且通过轴突、神经末梢与下一个神经元突触完成信号传递。



神经网络算法的基本思想

受人脑神经网络启发，Warren McCulloch 和 Walter Pitts 于 1943 年提出了 MP 神经元模型，MP 神经元模型与前述神经元的信号传递逻辑抑制。基本思路如下图所示：



神经网络算法的基本思想

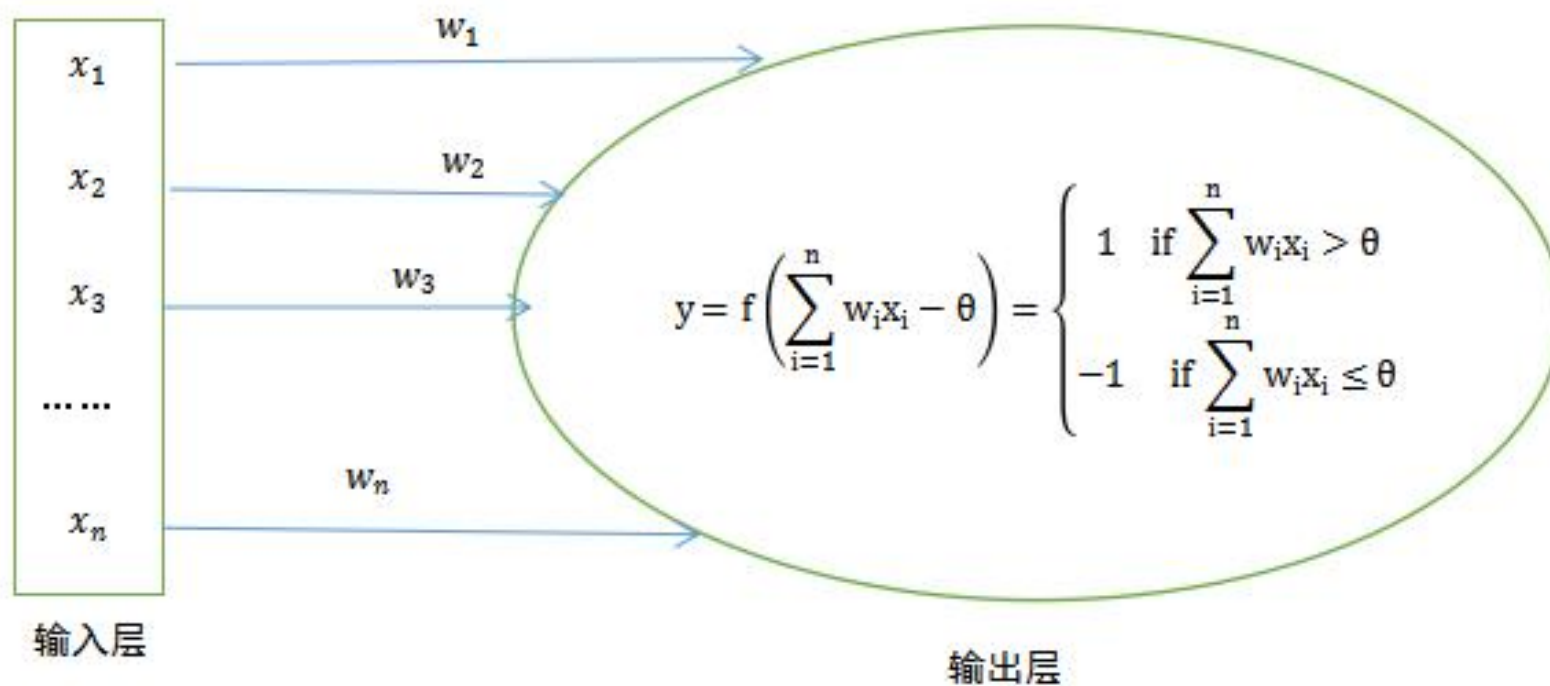
MP 神经元模型从左侧接受来自 x_i 的信号，每个 x_i 的信号权重为 w_i ，从而得到的信号值为 $\sum_{i=1}^n w_i x_i$ ，如果 $\sum_{i=1}^n w_i x_i$ 能够大于阈值 θ ，意味着接受信号突破了兴奋阈值 θ 的屏障，从而可以向下一神经元进行传递，如果 $\sum_{i=1}^n w_i x_i$ 小于等于阈值 θ ，意味着接受信号无法突破兴奋阈值 θ 的屏障，从而信号被抑制，无法继续向下一神经元进行传递。

$$y = f\left(\sum_{i=1}^n w_i x_i - \theta\right) = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i > \theta \\ 0 & \text{if } \sum_{i=1}^n w_i x_i \leq \theta \end{cases}$$

其中函数 $f()$ 被称为神经元激活函数，阈值 θ 也被称为偏置。上述 MP 神经元是基本模型，把很多个这样的神经元连接起来，就生成了神经网络。从上面所述可以看出，仿真模拟人脑神经网络只是 MP 神经元模型的直观解释，MP 神经元模型实质上就是包含多个参数的数学模型。但需要特别说明的是，MP 神经元模型不具备学习能力，在 MP 神经元模型中， w_i 和 θ 的值只能人为指定，而无法通过模型训练获得。

感知机

感知机（Perceptron Linear Algorithm, PLA）由Fran Rosenblatt于1957年提出。该算法使得MP神经元模型具备了学习能力，也被视为神经网络（深度学习）的起源算法。该算法的基本特点是模型由两层神经元构成，分别是输入层和输出层，但没有隐藏层，其中输入层为接受外部信号的层，输出层为MP神经元模型。



如上图所示，在感知机模型中，输入层为特征变量 x ， x_i 为特征变量 x 对应于第 i 个输入神经元的分量，输出层为响应变量 y 。

感知机

考虑一个二分类问题，假定响应变量 y 的取值只有 1 和 -1，在感知机模型中，如果 $\sum_{i=1}^n w_i x_i > \theta$ ，则感知机模型将响应变量 y 预测为 1；如果 $\sum_{i=1}^n w_i x_i < \theta$ ，则感知机模型将响应变量 y 预测为 -1；如果 $\sum_{i=1}^n w_i x_i = \theta$ ，则感知机模型将对响应变量 y 进行随机预测。

在上述规则下，如果 $y(\sum_{i=1}^n w_i x_i - \theta) > 0$ ，也就是说在下述两种情形时，感知机预测为准确：

$$\begin{cases} \left(\sum_{i=1}^n w_i x_i - \theta \right) > 0 \text{ 且 } y > 0 & \text{此时响应变量的预测值取值为 1，且实际值为 1} \\ \left(\sum_{i=1}^n w_i x_i - \theta \right) < 0 \text{ 且 } y < 0 & \text{此时响应变量的预测值取值为 -1，且实际值为 -1} \end{cases}$$

如果 $y(\sum_{i=1}^n w_i x_i - \theta) < 0$ ，也就是说在下述两种情形时，感知机预测为错误：

$$\begin{cases} \left(\sum_{i=1}^n w_i x_i - \theta \right) > 0 \text{ 且 } y < 0 & \text{此时响应变量的预测值取值为 1，但实际值为 -1} \\ \left(\sum_{i=1}^n w_i x_i - \theta \right) < 0 \text{ 且 } y > 0 & \text{此时响应变量的预测值取值为 -1，但实际值为 1} \end{cases}$$

感知机模型的基本思想是通过参数 w_i 和 θ 的充分调整，使得模型的错误分类为最少。从数学公式的角度看，就是要满足以下损失函数：

$$\underset{w, \theta}{\operatorname{argmin}} L(w, \theta) = - \sum_{j=1}^m \left[\left(\sum_{i=1}^n w_i x_{ij} - \theta \right) \times y_j \right]$$

其中假定特征变量 x 输入神经元的分量共有从 $i = 1 \dots n$ 合计 n 个，分类错误的响应变量 y 共有从 $j = 1 \dots m$ 合计 m 个， w 为权重向量 $(w_1, w_2, \dots, w_n)$ 。

前面我们在第 15 章中讲到，在求解约束优化问题时，梯度下降是最常采用的方法之一。损失函数的负梯度其实就是损失函数的导数的负数，数学公式为：

$$-g(x) = - \left\{ \frac{\partial L(y, F(x))}{\partial F(x)} \right\}$$

上述最优化函数的负梯度即为：

$$\begin{cases} -\frac{\partial L(w, \theta)}{\partial w} = \sum_{j=1}^m \sum_{i=1}^n x_i y_j \\ -\frac{\partial L(w, \theta)}{\partial \theta} = \sum_{j=1}^m y_j \end{cases}$$

感知机模型沿着损失函数负梯度方向迭代，使得损失函数越来越小，具体学习规则为：

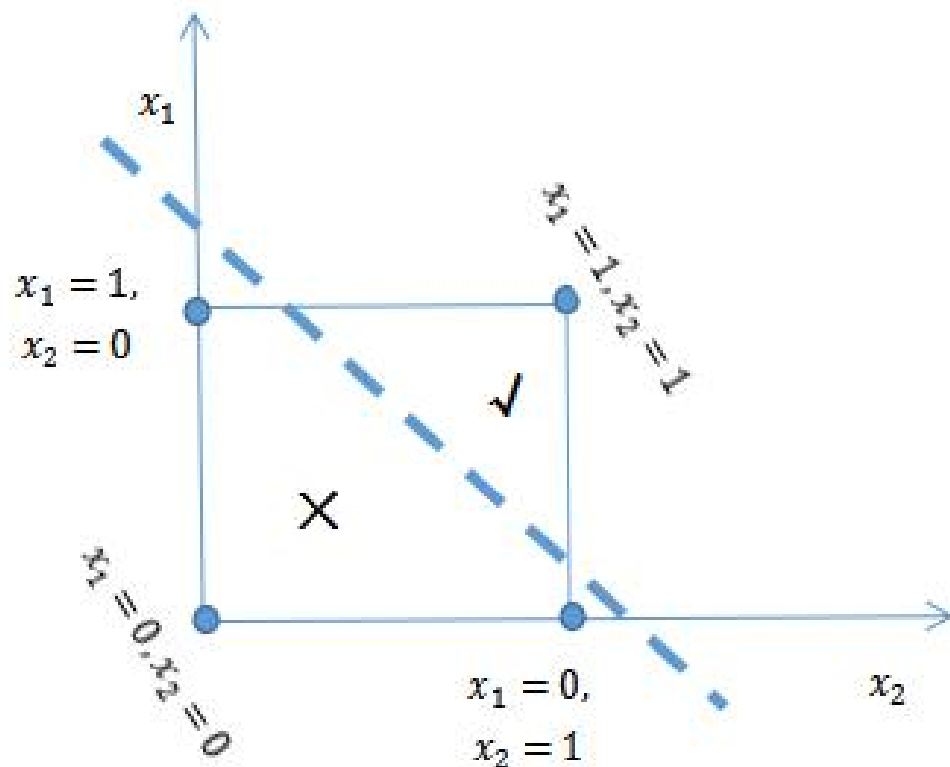
$$\begin{cases} w \leftarrow w + \eta \sum_{j=1}^m \sum_{i=1}^n x_i y_j \\ \theta \leftarrow \theta + \eta \sum_{j=1}^m y_j \end{cases}$$

其中的 η 为学习率，取值范围为 $(0,1)$ ，设置为一个较小的正数。因为我们仅考虑分类错误的响应变量，所以当样本示例分类正确时，参数 w, θ 将不作调整，而针对分类错误的样本示例，在感知机训练的过程中，会根据错误的程度进行参数调整，通过参数 w, θ 的不断迭代，使得分类超平面不断向错误分类的样本观测值进行移动，从而模型偏差越来越小，直至实现正确分类。

不难发现，对于感知机模型而言，只有输出层通过神经元激活函数进行了实质性数据处理，所以这也决定了其学习能力非常有限。如果数据集是线性可分的，或者说存在一个线性超平面将数据有效分离（当然这一超平面并不唯一），那么可以证明感知机一定会收敛，进而可以通过学习来求得参数 w 和 θ ，但不能忽视的是，参数 w 和 θ 的初始值不同，得到的超平面也就会不同，无法确保所得到的超平面为最优超平面。而如果数据集无法做到线性可分，或者说无法找到一个线性超平面将数据有效分离，那么感知机将无法实现收敛，使得算法无法实现。感知机算法仅针对线性可分数据有效，其决策边界，也就是前面提到的分离超平面，为线性函数，无法用于解决非线性决策边界问题。

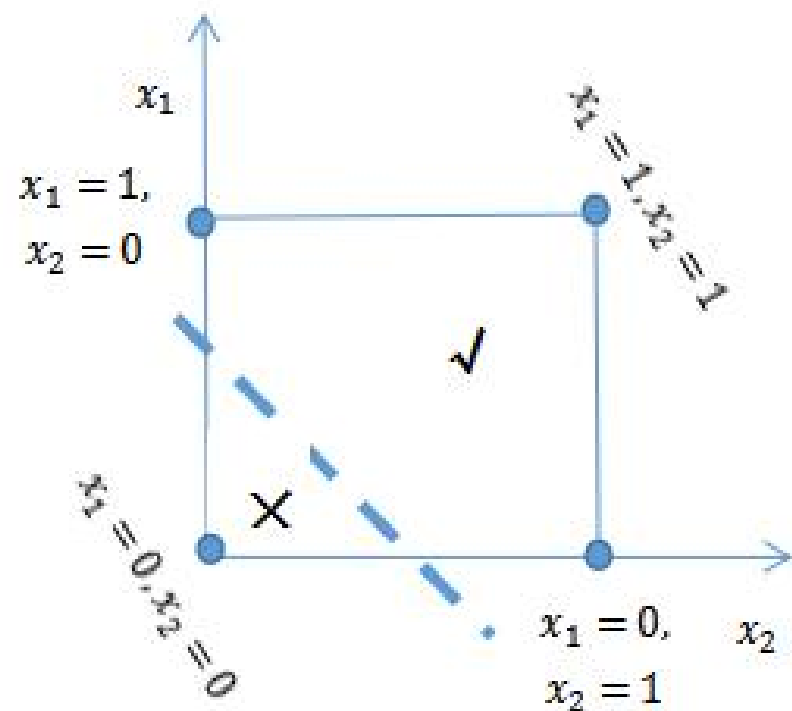
感知机

比如针对逻辑运算，“和”“或”以及“非”均为线性可分问题，感知机模型都可以较好的解决。其中“和”问题如下图所示， x_1 、 x_2 为参与逻辑运算的两个变量，分别位于纵轴和横轴，两个变量都是取值为 0 时表示为假，取值为 1 时表示为真。在“和”问题中，只有 x_1 、 x_2 两个值均取值为 1（均取真）时，其合并结果才为真，其他情形均为假。虚线为分离超平面，可以发现虚线左侧的点均为“假”（×号区域），右侧的点为“真”（√号区域）。

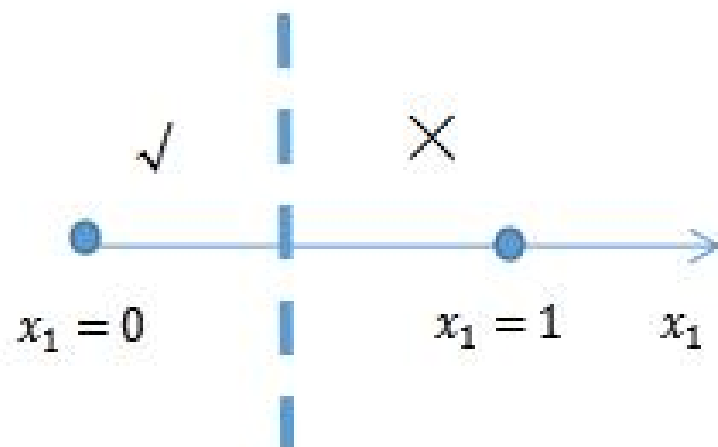


感知机

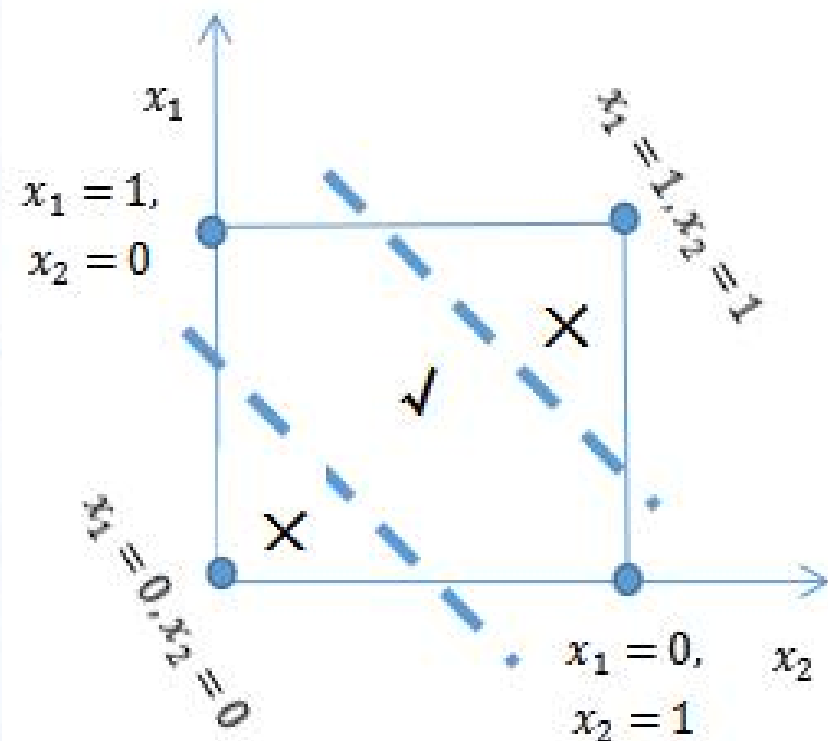
感知机模型也可以较好的解决逻辑运算中的“或”问题。在“或”问题中，只要 x_1 、 x_2 两个值中有一个取值为1（取真）时，其合并结果就为真，只有两个值均取值为0（取假）时才为假。如下图所示，虚线左侧的点为“假”（×号区域），右侧的点均为“真”（√号区域）。



感知机模型也可以较好的解决逻辑运算中的“非”问题，针对变量 x_1 ，如果其取值为真，则其“非”运算为假，反之则反是。如下图所示，虚线左侧的点为“假”（×号区域），右侧的点为“真”（×号区域）。



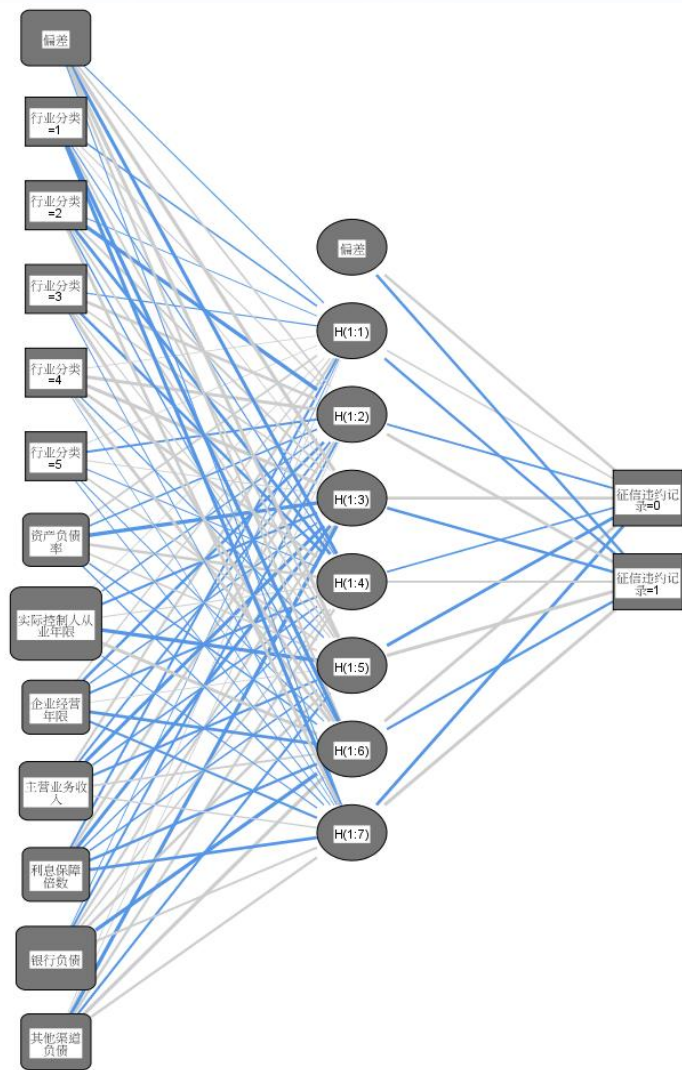
而逻辑运算中的“异或”问题是一种非线性可分问题，感知机模型就不能够解决。如果 x_1 、 x_2 两个值不相同，则异或结果为 1；如果 x_1 、 x_2 两个值相同，异或结果为 0。如下图所示，无法通过一条虚线（分离超平面）将 $\times$ 号区域和 $\checkmark$ 号区域分开，无法达到分类目的。



多层感知机

前面我们提到，感知机无法解决非线性可分问题，为了解决这一问题，我们需要将两层神经元扩展至更多层，也就是说在感知机的输入层和输出层之间加入隐藏层，隐藏层与输出层一样，都可以通过神经元激活函数进行实质性数据处理。比如一个包含隐藏层的神经网络如下图所示（图片来源：《SPSS统计分析商用建模与综合案例精解》

（杨维忠、张甜著，清华大学出版社）第4章商业银行授信客户信用风险评估）。



隐藏层激活函数：双曲正切
输出层激活函数：Softmax

该神经网络最左侧为输入层，包括“行业分类”、“资产负债率”、“实际控制人从业年限”、“企业经营年限”、“主营业务收入”、“利息保障倍数”、“银行负债”、“其他渠道负债”等特征变量，不包括激活函数；中间为隐藏层，“隐藏层”包含无法观察的节点或单元，每个隐藏单元的值都是“输入层”中特征变量的激活函数，函数的确切形式部分取决于具体的神经网络类型，本例中激活函数为双曲正切函数；右侧为输出层响应变量“征信违约记录”，每个响应都是隐藏层的激活函数，本例中激活函数为Softmax函数。

|| 多层感知机

在该神经网络中，从第一层开始，除最后一层，每一层的神经元均与下一层的所有神经元产生联系，而同层神经元之间均没有产生联系，也不存在神经元之间的跨层联系，这种典型的神经网络被称为是“全连接神经网络（Fullyconnectedneuralnetwork,FCNN）”，而从数据流动方向来看，由于整体上数据从左到右逐层传递，信号从输入层到输出层单向传播，所以也被称为“多层前馈神经网络（multilayerfeedforwardneuralnetwork）”，由于多层前馈网络的训练经常采用误差反向传播算法（下文中将进行讲解），所以多层前馈网络又被称为 **BP 网络（BackPropagation）**。

需要特别说明的是，“多层感知机”指的是包含了隐藏层的感知机，隐藏层可以有許多层，或者说隐藏层个数可以大于或等于 1，在有多个隐藏层情况下，下一个隐藏层的每个单元都是上一个隐藏层单元的激活函数，最后“输出层”的每个响应都是最后一个隐藏层单元的一个函数。如果 神经网络的隐藏层很多，那么就被称为“**深度神经网络（DeepNeuralNetworks,DNN）**”，对于“深度神经网络”的训练就是“深度学习（Deeplearning）”。

神经网络中的模型参数，是神经元模型中的连接权重以及每个功能神经元的阈值，神经网络算法的本质就是基于训练示例，不断调整神经元之间的连接权重和阈值，即持续优化参数 w 和 b 来实现的。

在多层感知机中，我们完全可以不再像感知机模型那样受数据集线性可分的约束，不仅可以解决非线性问题，也完全可以得到非线性的决策边界，这一点通过引入两个及以上非线性的激活函数（至少一个隐藏层以及输出层）来实现，如果引入的激活函数依旧为线性，也不可能得到非线性的决策边界。

|| 神经元激活函数

无论是何种神经网络算法，其能够有效发挥作用的根由是因为激活函数的存在。如果我们没有设置激活函数，那么神经网络的各层智慧根据神经元连接权重参数和阈值参数进行线性变化，即使设置再多隐藏层，线性变换的结果仍然是线性，而无法解决任何复杂的非线性问题。为了能够处理非线性问题，就需要设置激活函数，激活函数必须是非线性的，通过激活函数的非线性变换，达到解决复杂学习问题的目的。常用的神经元激活函数包括：S型函数、双曲正切函数、ReLU函数、泄露ReLU函数和软加函数。

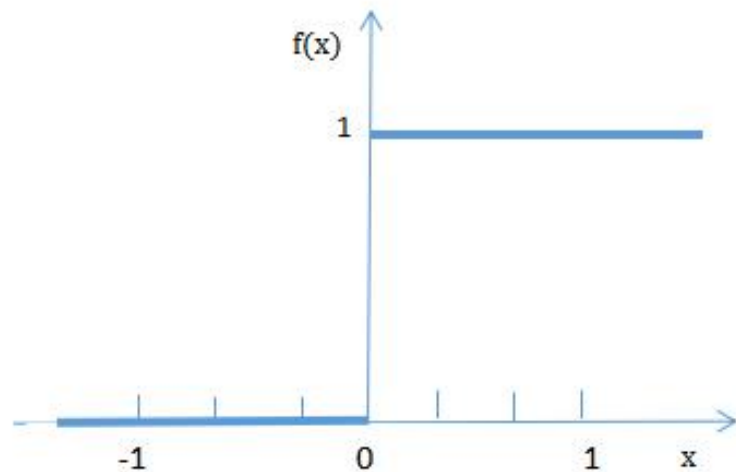
神经元激活函数

一、S型函数 (sigmoid 函数)

在 17.1.2 所述的感知机模型中，使用的激活函数为阶跃函数 (sgn)，其数学公式为：

$$\text{sgn}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

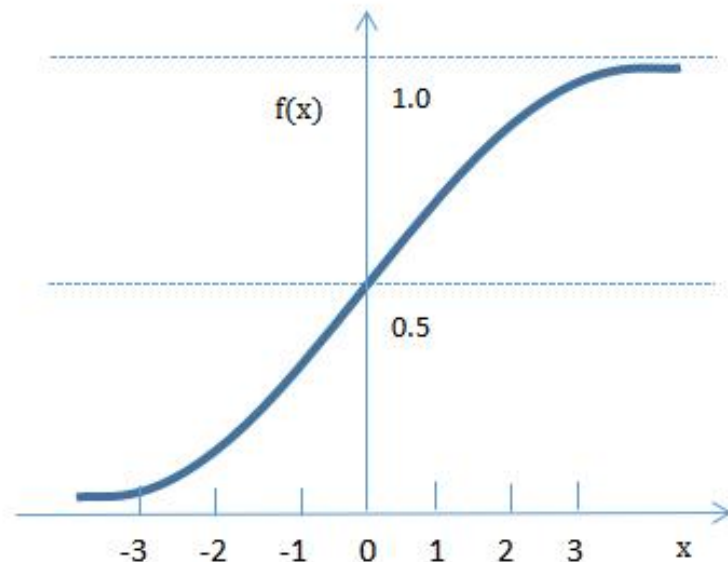
即当输入值达到阈值是取值为 1，达不到阈值时取值为 0（前面讲解感知机时定义的是 y 取值为 1、-1，但原理与此处的 1、0 二元取值完全相同）。这一函数虽然较为理想，但是不连续、不光滑，无法实现连续可导，在求解时会遇到较大的障碍。



所以在实务中不用阶跃函数，而是使用 S 型函数、双曲正切函数、ReLU 函数、泄露 ReLU 函数和软加函数。广义的 S 型函数不是某一个函数，而是指某一类形如“S”的函数，都可以称为 S 型函数。S 型函数的典型代表是逻辑函数，也被称为狭义的 S 型函数，其数学公式为：

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

S 型函数也被称为挤压函数，较大范围区间取值的输入值，可以通过 S 型函数 (sigmoid 函数) 进行压缩，将 x 值转化为接近 0 或 1 的值，使得输出值在 (0, 1) 范围内。S 型函数输入值在 x=0 附近变化很陡峭，近似为线性，但是在当输入值在两端时则对其进行显著的挤压，右端的输入值被压缩到接近于 1（但小于 1），左端的输入值为压缩到接近于 0（但大于 0）。



|| 神经元激活函数

sigmoid 函数虽然具有连续可导的优点，但也有一定的弊端。sigmoid 函数在定义域内两侧导数逐渐趋近于 0，是一种双向软饱和激活函数（函数自变量 x 的变动引起其函数值 $f(x)$ 的变动很小，那么就称其为饱和）。当输入值靠近两端时，大的输入值的改变对应了小的输出值的改变，输出值的梯度较大（小小改变即可），而输入值的梯度较小（大大改变才行），随着层数的增多，反向传递（从输出到输入）的残差梯度会越来越小，从数学上即表现为 sigmoid 函数的导数趋向于 0，也就是说出现了梯度消失（Vanishing Gradient）的问题，使得 BP 算法（即误差反向传播算法）中依赖的梯度下降法失效，减缓收敛速度。

即使靠近输出层的神经网络已经获得了最优参数，但由于不能有效传递到输入层，靠近输入层的神经网络并没有得到很好的训练，仍旧处于参数随机初始化状态，导致靠近输入层的隐藏层发挥不出作用。通常来说，sigmoid 网络在 5 层之内就会产生梯度消失现象。机器学习中与梯度消失对应的概念是梯度爆炸（Vanishing Explode），会导致神经网络无法收敛。

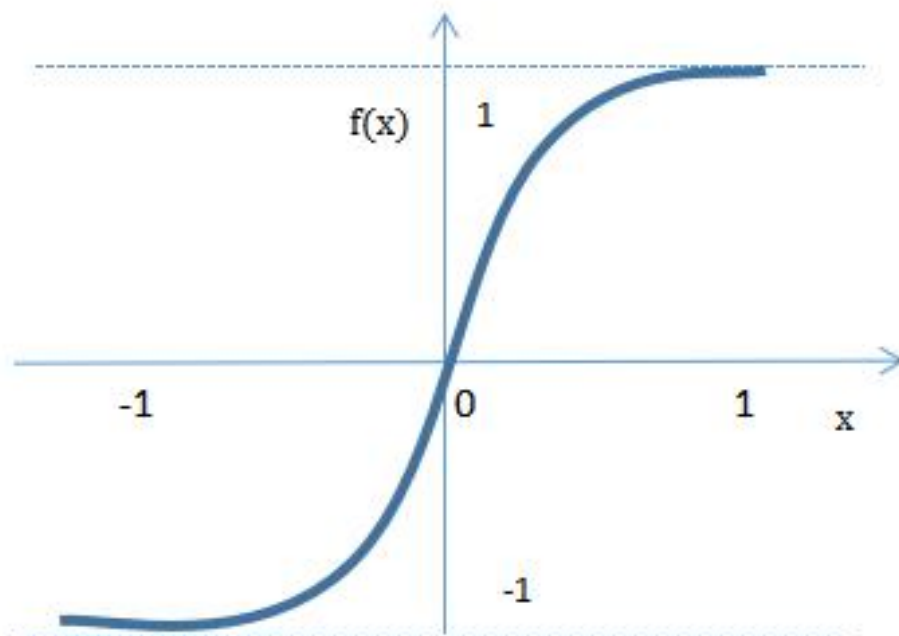
|| 神经元激活函数

二、双曲正切函数 (tanh 函数)

双曲正切函数 (tanh 函数) 也属于广义的 S 型函数, 其贡献在于将 Sigmoid 函数的取值范围拉长到 $(-1, 1)$ 区间, 实现输出值以 0 为中心, 解决了 Sigmoid 函数的不以 0 为中心的输出问题 (因为 Sigmoid 函数的取值范围为 $(0, 1)$, 以 0.5 为中心输出), 但同样没有解决梯度消失的问题 (理由同 sigmoid 函数)。

其数学公式为:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



三、ReLU函数(Rectified Linear Units 函数)

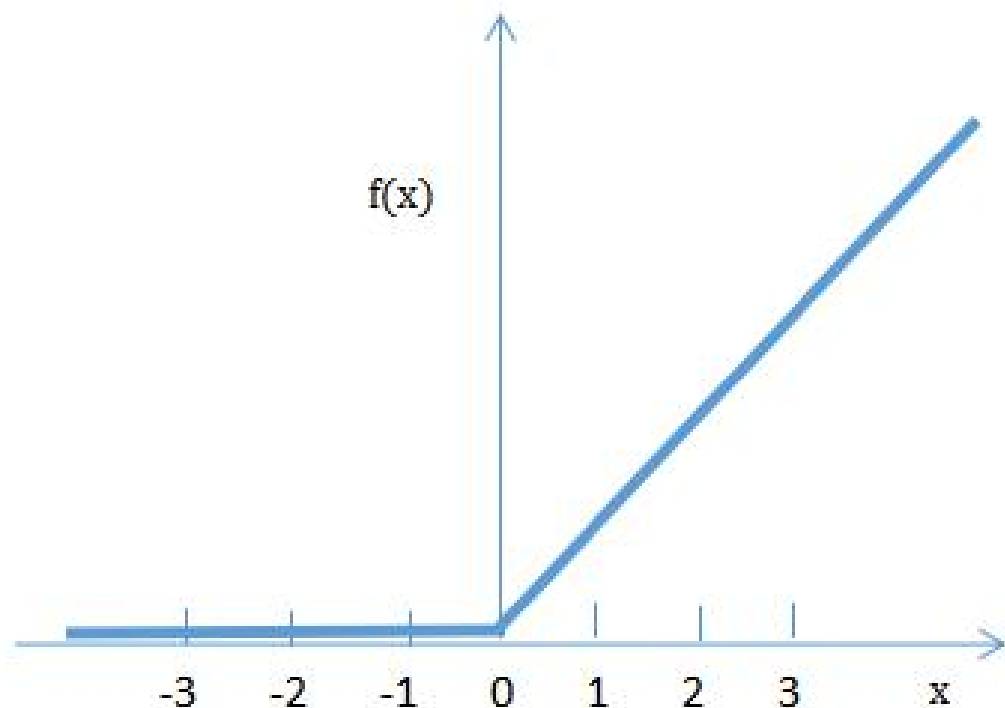
ReLU函数，也称线性整流函数、修正线性单元，是一种后来才出现的激活函数，也是目前最常用的默认选择激活函数。相对于S型函数，ReLU函数具有左侧单侧抑制、相对宽阔的右侧兴奋边界以及稀疏激活性三个特点。线性整流函数在一定程度上缓解了前述S型函数中存在的梯度消失问题，实现了更加有效率的梯度下降以及反向传播，而且被认为有一定的生物学原理，这样就带来了神经网络的稀疏性，并且减少了参数的相互依存关系，有效缓解了过拟合问题，所以广泛应用于深度神经网络学习，尤其是在图像识别领域备受推崇。

从数学公式来看，线性整流函数指代数学中的斜坡函数，即 $f(x) = \max(0, x)$ ，具体为：

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

|| 神经元激活函数

当输入值 $x \geq 0$ ，ReLU 函数即为线性函数，输入值等于输出值，对应生物学中人脑的兴奋度可以很高、不设限；而当输入值 $x < 0$ 时，输出值均为 0，对应生物学中人脑的单侧抑制。不难发现，当输入值小于 0 时，ReLU 函数导数趋向于 0；当输入值大于等于 0 时，ReLU 函数导数保持为常数 1，也就是说当 $x < 0$ 时，ReLU 函数硬饱和，而当 $x \geq 0$ 时，则不存在饱和问题。所以 ReLU 能够在 $x \geq 0$ 时保持梯度不衰减，缓解了前述 S 型函数中存在的梯度消失问题。



|| 神经元激活函数

2001 年，Attwell 等人推测人脑神经元工作方式具有稀疏性，神经元同时只对输入信号的少部分选择性响应，大量信号被刻意的屏蔽了，这样可以提高学习精度，从而更好更快地提取稀疏特征；2003 年 Lennie 等人估测人脑同时被激活的神经元只有 1%~4%，或者说人脑中同时处于兴奋状态的神经元占比只有 1%~4%，进一步表明了神经元工作的稀疏性。从这种角度，ReLU 函数因为会使一部分神经元的输出为 0，所以能够产生较好的稀疏性，更符合人脑生物学规律。

但 ReLU 函数也存在一些不足，不难发现，与 sigmoid 类似，ReLU 函数同样不以 0 为中心输出，输出均值会大于 0，会导致下一层偏置偏移从而影响梯度下降的效果。并且随着训练的推进，部分输入会落入硬饱和区（即 $x < 0$ ），ReLU 函数导数趋向于 0，导致对应权重无法更新，这个神经元再也不会对任何数据有激活现象了，也就说无论输入值是什么，输出值都是 0，这种现象也被称为“神经元死亡”，尤其是当学习率设置的比较高时，这种问题会表现的更为突出。ReLU 的偏置偏移现象和神经元死亡会共同影响神经网络的收敛性。

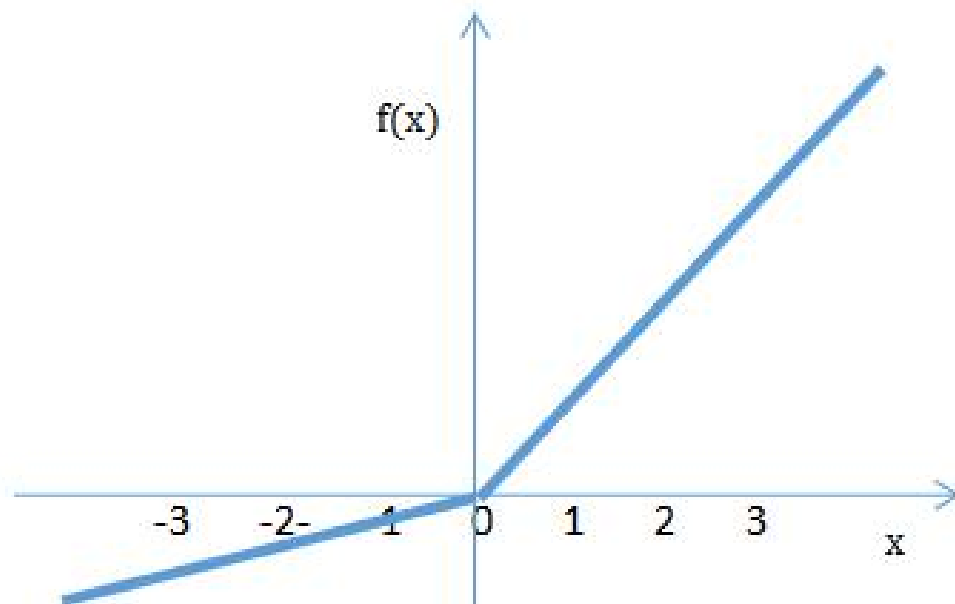
|| 神经元激活函数

四、泄露 ReLU 函数 (leakyReLU 函数, LReLU)

为了解决上述“神经元死亡”问题，或者说针对 ReLU 函数中存在的在 $x < 0$ 时的硬饱和问题，学者们对 ReLU 函数做出了改进，提出了泄露 ReLU 函数，即 $f(x) = \max(x, \alpha x)$ ，其中 α 的取值范围为 $(0, 1)$ ，具体为：

$$\text{LReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases}$$

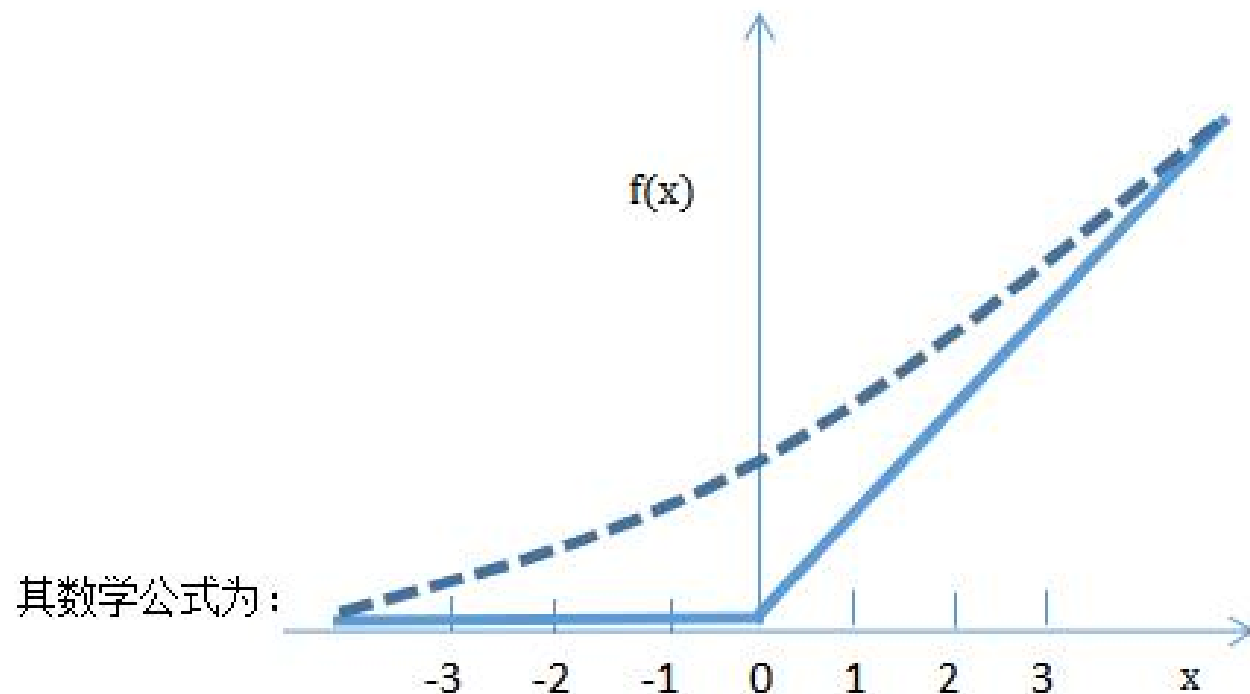
相对于 ReLU 函数，在泄露 ReLU 函数中，当 $x < 0$ 时，函数不再实施硬饱和，而是保持一个较小的非零的梯度 α 继续更新参数，从而有效避免了“神经元死亡”问题。



|| 神经元激活函数

五、软加函数 (Softplus 函数)

下图中的虚线为软加函数，实线为 ReLU 函数，软加函数一方面比 ReLU 函数更加光滑一些，另一方面也有效解决了左侧 $x < 0$ 时的硬饱和问题（导数不再一直为 0）。



不难发现，与 ReLU 函数相比，软加函数同样具有左侧单侧抑制、相对宽阔的右侧兴奋边界两个特点，但是由于其导数永远为正（可通过上述数学公式计算），所以不具备稀疏激活性的特点。

|| 误差反向传播算法（BP算法）

误差反向传播算法（BP算法）最早由PaulJ.Werbos于1974年提出，但在当时并未引起足够重视，后由DavidEverettRumelhart（1986）重新发明了该算法，是目前应用最广泛的神经网络算法。

针对感知机模型，因为没有包含隐藏层，结构相对简单，所以可以非常便捷的直接使用梯度下降法计算损失函数的梯度。但是在多层感知机模型中，上一层的输出是下一层的输入，要在神经网络中的每一层计算损失函数的梯度将变得非常复杂。为了有效解决这一问题，可以使用BP算法。

一个相对明确的事实是，在多层神经网络中，距离输出层越近，或者说越靠近网络后端（右侧），其导数越容易计算。BP算法充分考虑这一特点，利用微积分中的链式法则，反向传播损失函数的梯度信息，通过从后往前遍历一次神经网络，从而计算出损失函数对神经网络中所有模型参数的梯度，系统解决了多层神经网络隐藏层连接权重学习问题。

|| 误差反向传播算法（BP算法）

BP 算法的本质是以神经网络的误差平方为目标函数、采用梯度下降法来计算目标函数的最小值。其操作步骤为：

1、数据预处理

将所有特征变量进行归一化（最大值设为 1，最小值设为 0，其他数据按比例均匀压缩）或标准化（减去均值，再除以标准差）处理，以消除特征变量的量纲差距对神经网络权重参数取值的影响。

2、随机初始化参数

在 $(0,1)$ 区间内随机初始化神经网络中的所有参数，包括神经元之间的连接权重 w 和阈值 θ 。之所以采取随机初始化而不是将所有参数都同质化取值 0 或 1 的方式，是因为随机初始化有利于保持不同神经元之间的差异性，避免趋同，随机初始化的具体实现方式一般是从 $[-0.7,0.7]$ 的均匀分布或 $N(0,1)$ 的标准正态分布中随机抽样。

误差反向传播算法（BP算法）

3、进行正向传播计算输出值

基于训练样本，将特征变量的数据输入到神经网络的输入层，然后经过隐藏层，最后达到输出层，根据所有初始化参数计算计算响应变量的输出值。

在正向传播过程中，假设第 m 个隐藏层的第 i 个神经元的输出值（激活值）为 $T_i^{(m)}$ ，第 m 个隐藏层的第 i 个神经元共与第 $m - 1$ 个隐藏层中的 K 个神经元发生联系，则有

$$T_i^{(m)} = f \left\{ \sum_{j=1}^K w_{ji}^{(m)} T_j^{(m-1)} \right\} \equiv f(u_i^m)$$

公式中 $f()$ 为激活函数， $T_j^{(m-1)}$ 为第 $m - 1$ 个隐藏层中的第 j 个神经元的输出值， $w_{ji}^{(m)}$ 为第 $m - 1$ 个隐藏层中第 j 个神经元的输出值对 $T_i^{(m)}$ 的权重。 $u_i^m \equiv \sum_{j=1}^K w_{ji}^{(m)} T_j^{(m-1)}$ 为施加激活函数之前的净输入函数。

误差反向传播算法 (BP算法)

4、进行反向传播计算误差和偏导数

计算响应变量输出结果和实际值之间的误差,将该误差从输出层反向传播,计算每一层的误差,即从输出层传播到最后一个隐藏层,然后依次传播,最终从第一个隐藏层传播至输入层,在反向传播的过程中,计算每一层的偏导数。

在反向传播过程中,设神经网络的损失函数为 L , 将 L 对权重参数 $w_{ji}^{(m)}$ 求偏导, 因为 $w_{ji}^{(m)}$ 仅通过净输入函数 u_i^m 影响损失函数 L , 所以根据微积分中的链式法则, 即有:

$$\frac{\partial L}{\partial w_{ji}^{(m)}} = \frac{\partial L}{\partial u_i^m} \times \frac{\partial u_i^m}{\partial w_{ji}^{(m)}} = \frac{\partial L}{\partial u_i^m} \times T_j^{(m-1)} \equiv E_i^m T_j^{(m-1)}$$

其中 $E_i^m \equiv \frac{\partial L}{\partial u_i^m}$ 为误差。由于误差是反项传播的, 所以 u_i^m 通过第 $m+1$ 个隐藏层中的所有神经元的净输入函数 u_k^{m+1} 对损失函数形成影响, 假设第 $m+1$ 个隐藏层中共有 G 个神经元, 所以根据微积分中的链式法则, 即有:

$$E_i^m \equiv \frac{\partial L}{\partial u_i^m} = \sum_{k=1}^G \frac{\partial L}{\partial u_k^{m+1}} \times \frac{\partial u_k^{m+1}}{\partial u_i^m} = \sum_{k=1}^G E_k^{m+1} \times \frac{\partial u_k^{m+1}}{\partial u_i^m}$$

而根据前述公式定义,

$$\begin{aligned} E_k^{m+1} &\equiv \frac{\partial L}{\partial u_k^{m+1}} \\ u_k^{m+1} &= \sum_{j=1}^K w_{jk}^{(m+1)} f(u_j^m) \\ \frac{\partial u_k^{m+1}}{\partial u_i^m} &= w_{jk}^{(m+1)} f'(u_j^m) \end{aligned}$$

则有:

$$E_i^m = \sum_{k=1}^G E_k^{m+1} w_{jk}^{(m+1)} f'(u_j^m) = f'(u_j^m) \sum_{k=1}^G E_k^{m+1} w_{jk}^{(m+1)}$$

该公式证明了第 m 个隐藏层的误差 E_i^m 是第 $m+1$ 个隐藏层的误差 E_k^{m+1} 的函数, 从而可以采用递归法计算误差 E_i^m , 并将其代入公式 $\frac{\partial L}{\partial w_{ji}^{(m)}} = \frac{\partial L}{\partial u_i^m} \times \frac{\partial u_i^m}{\partial w_{ji}^{(m)}} = \frac{\partial L}{\partial u_i^m} \times T_j^{(m-1)} \equiv E_i^m T_j^{(m-1)}$, 即可得到偏导数 $\frac{\partial L}{\partial w_{ji}^{(m)}}$ 。

|| 误差反向传播算法（BP算法）

5、基于梯度下降策略优化参数

神经网络作为一种机器学习算法，其训练方法也是在参数空间使用梯度下降法，使损失函数最小化。具体操作为：给定的学习率 η ，基于梯度下降策略，以总损失函数最小化为目标，更新神经元之间的连接权重 w 和阈值 θ 。对于给定的学习率 η ，学习率 η 用于控制算法每轮迭代中的步长，设置过大会容易引起震荡，设置过小则会容易造成收敛速度过慢，需要合理设置。

上面的描述本质上就是求解如下最优化问题：

$$\underset{W}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n L[y_i, F(x_i; W)]$$

公式中的 n 为样本容量， $F(x_i; W)$ 是一个多层前馈神经网络模型，由于实务中大多数数据集是不满足线性可分条件的，所以 $F(x_i; W)$ 也大概率是一个复杂的非线性函数，以满足解决问题的需要。 $L[y_i, F(x_i; W)]$ 为损失函数，对于回归问题而言，一般使用“平方损失函数”（详见第 15 章中相关介绍），最小化训练集的均方误差；对于分类问题而言，二分类问题一般使用“逻辑损失函数”，多分类问题一般使用“交叉熵损失函数”（详见第 15 章中相关介绍）

|| 误差反向传播算法（BP算法）

传统梯度下降法下，在求解损失函数的梯度向量时，需要针对每个样本示例的损失 $L[y_i, F(x_i; W)]$ 分别求解各自的梯度向量，然后进行加总求平均。由于传统的梯度下降法针对的是整个样本数据集，通过对所有的样本的计算来求解梯度的方向，所以也被称为**批量梯度下降法 BGD(Batch Gradient Descent)**，其优点在于能够获得全局最优解，易于并行实现，缺点在于当样本数据很多时，计算量开销大，计算速度慢。

可以合理预期的是，如果样本容量 n 较大，那么由之伴生的参数及计算量之大，将成为算法不可承受之重，所以传统的梯度下降法不能很好的满足大样本数据集的实际计算需求。为了解决这一问题，有以下方法可供选择：

（1）随机梯度下降法（stochastic gradient descent, SGD）

随机梯度下降法的基本思想是每次无放回的随机抽取一个样本示例，并计算该样本示例的负梯度向量，并沿着负梯度方向使用给定的学习率 η 进行参数更新。

$$W \leftarrow W - \eta \frac{\partial L[y_i, F(x_i; W)]}{\partial W}$$

随机梯度下降法的优点是计算速度快，因为每次只需要计算一个样本示例的负梯度向量，缺点是收敛性能不好，因为单一样本示例的负梯度方向难以代表整体样本示例全集的负梯度方向，从而在收敛过程中会出现较多的曲折反复，当然如果从长期趋势来看，该方法依旧会朝着整体损失函数最小值的方向进行收敛。

|| 误差反向传播算法（BP算法）

（2）小批量梯度下降法（mini-batch Gradient Descent, MBGD）

小批量梯度下降法的基本思想是每次无放回的随机抽取多个样本示例（比如 M ， M 的值通常不大，所以被称为小批量），并计算这些样本示例的负梯度向量的平均值，沿着负梯度方向使用给定的学习率 η 进行参数更新。

$$W \leftarrow W - \eta \frac{1}{M} \sum_{i=1}^M \frac{\partial L[y_i, F(x_i; W)]}{\partial W}$$

相对于随机梯度下降法，小批量梯度下降法把数据分为若干批，基于一组样本示例而不是单一样本示例来更新参数，从而在一定程度上解决了随机梯度下降法负梯度方向由单一样本决定导致容易跑偏的问题，较好的减少了随机性。

|| 误差反向传播算法（BP算法）

对比批量梯度下降法、随机梯度下降法、小批量梯度下降法三种方法，不难发现其计算方法的核心思想基本一致，**差别仅在于每次计算负梯度向量、更新参数所依据的样本示例规模**，如果仅随机抽取一个样本示例，就是随机梯度下降法；如果抽取一组小等规模的样本示例，就是小批量梯度下降法；如果基于全部样本示例进行计算，就是批量梯度下降法。在具体应用时，如果是针对小样本数据集，批量梯度下降法是首选；而如果针对大样本数据集，则小批量梯度下降法是首选，具体批量规模（随机抽取样本示例的个数）常用的包括 32、64、128、256，**在深度学习及复杂机器学习中，基本上都是使用小批量梯度下降法**。当然随机梯度下降法也绝非一无是处，多用于支持向量机、逻辑回归等凸损失函数下的线性分类器学习，已成功应用于文本分类和自然语言处理中经常遇到的大规模和稀疏机器学习问题。

6、算法迭代至收敛。迭代上述算法（正向传播计算输出结果-反向传播计算误差-基于梯度下降策略优化参数），直至满足收敛准则，训练停止。

|| 万能近似定理及多隐藏层优势

Hornik在1989年证明了万能近似定理(Universal approximation theorem): 只需一个包含足够多神经元的隐层, 一个前馈神经网络就能以任意精度逼近任意复杂的连续函数(Hornik et al., 1989; Cybenko, 1989)。也就是说, 即使前馈神经网络只有一个隐藏层, 但只要该隐藏层的神经元数量足够多, 就能够表示出任意连续函数。

万能近似定理充分说明了神经网络算法功能的强大, 可以作为万能函数来使用。但万能近似定理只是说可以表示出任意连续函数, 并未说怎么找到相应的神经网络, 也没有说到底需要设置多少个神经元是恰当的。而针对复杂问题, 我们更倾向于通过设置多个隐藏层, 而不是通过仅设置一个隐藏层、大幅增加神经元的方式。这是因为, 一方面神经网络中函数估参占用的计算资源通常呈指数增长, 如果神经元数量过多将从资源层面无法实现, 另一方面使用单隐藏层虽然可以表示任意复杂连续函数, 但也很可能导致模型的泛化能力不足。

|| 万能近似定理及多隐藏层优势

神经网络本质上也是类似于拟合函数的过程，复杂程度取决于模型的形式和参数的数量。虽然根据万能近似定理，仅有一个隐藏层的神经网络，就能拟合任意复杂的连续函数，但是设置多个隐藏层的深层网络可以用少得多的神经元去拟合同样的函数，浅层网络如果想要达到同样的计算结果，需要指数级增长的神经元数量才能达到。或者说，针对复杂问题，由于BP算法的存在，为达到既定计算结果，单一隐藏层通过神经元数量指数级增长的计算成本，在大多数情况下都要显著高于增加隐藏层带来的模型复杂度的增加成本。当然，这一经验性结果并不绝对，而且在隐藏层的具体设置个数方面目前也没有权威性指导，需要结合实际研究问题，采用试错法进行探索。

|| 万能近似定理及多隐藏层优势

所以在实务中，具有多个隐藏层的深度神经网络应用更为广泛，一般情况下，在深度神经网络的诸多隐藏层中，前面的隐藏层先学习一些低层次的简单特征，后面的隐藏层把简单的特征结合起来，去探索更加复杂的特征。比如针对人脸识别问题，前面的隐藏层首先将拍摄到的人脸图片提取出简单的轮廓特征，中间的隐藏层将简单的轮廓特征组织成眼睛、鼻子等器官特征，后间的隐藏层再将简单的轮廓特征组织成人脸图片，形成完整的面部特征，完成机器学习过程。

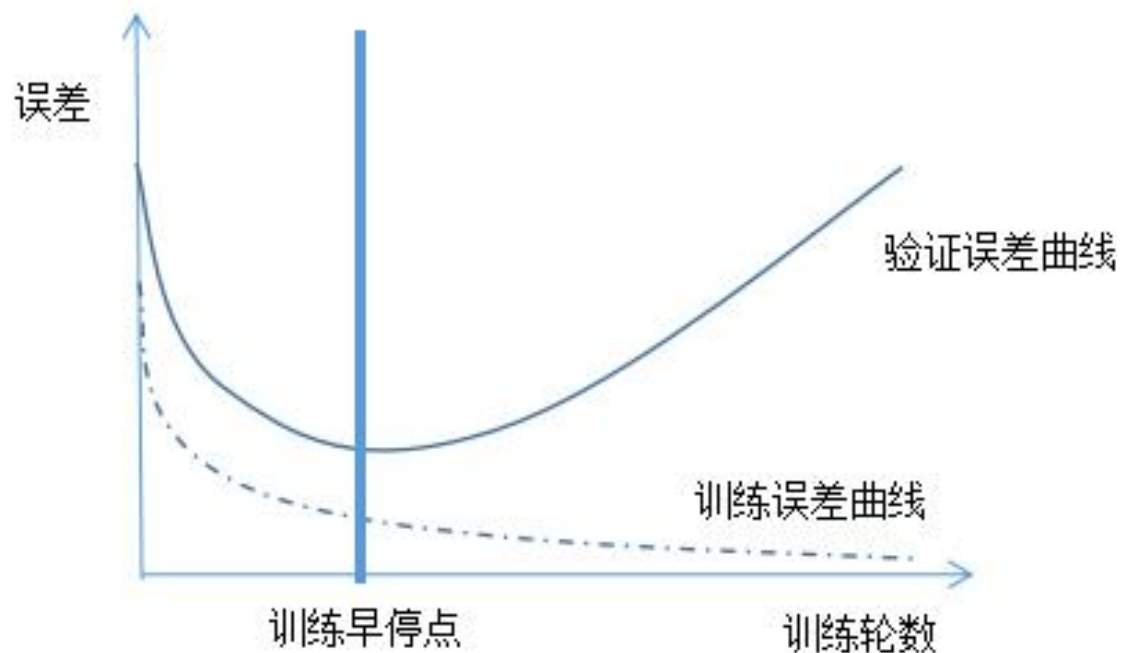
BP算法过拟合问题的解决

前面我们通过BP算法和万能近似定理见证了前馈神经网络隐藏层的强大，但也正是由于其强大，在基于训练样本拟合模型时容易出现过拟合的情况，导致模型不能很好的泛化到测试样本。为有效解决这一问题，有两种方法可供选择。

BP算法过拟合问题的解决

一、早停 (EarlyStopping)

早停方法的实现路径是：将样本示例全集分为三部分：训练样本、验证样本和测试样本；训练样本用来根据前述 BP 算法拟合前馈神经网络模型、计算训练误差，由于训练会越来越充分，所以训练误差会不断下降；验证样本用来基于训练样本得到的前馈神经网络模型独立计算误差，由于拟合的模型是基于训练样本得到的，所以验证误差与模型的泛化能力走势相同，会呈现先下降再上升的趋势，当验证误差停止下降、开始上升时，训练停止，这也就是所谓的“早停”；将停止训练后的模型应用到测试样本，计算测试误差，作为模型最终性能的度量。早停方法如下图所示：



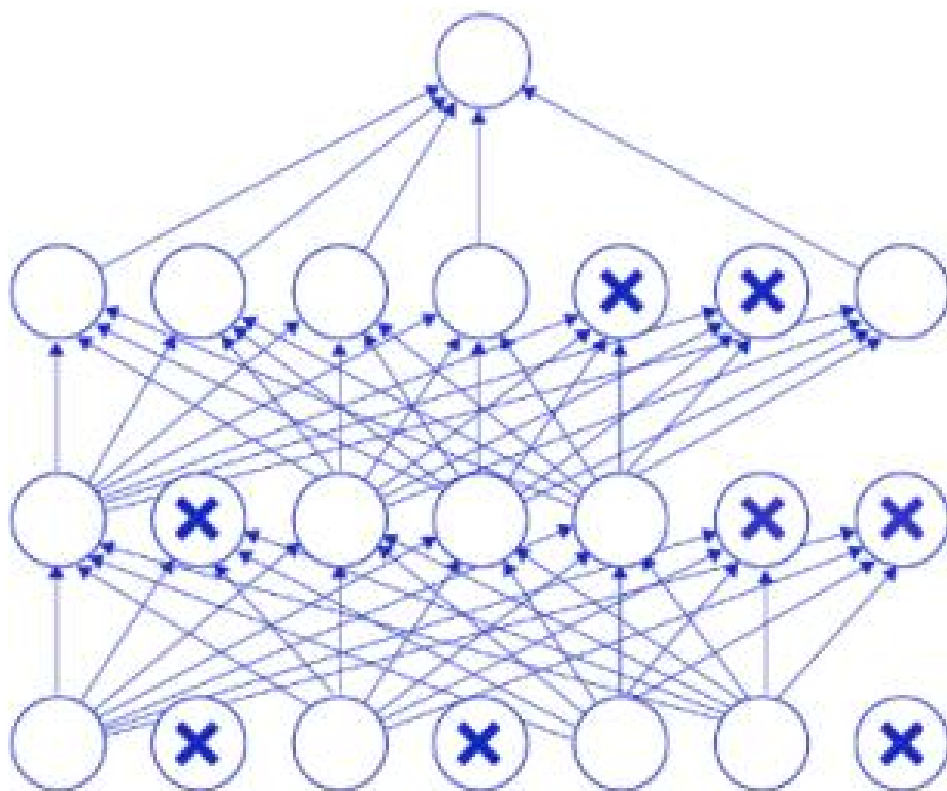
BP算法过拟合问题的解决

二、丢弃 (Dropout)

丢弃方法与装袋法的思想类似，本质上是通过随机隐藏神经元的方式训练不同的网络，然后针对得到的不同的神经网络取平均。该方法的实现路径是：在基于训练样本对神经网络模型进行训练时，每次训练时通过随机让一些神经元输出值取值为0的方式随机丢弃掉（也可以理解为暂时隐藏起）一些神经元，或者说人为断开这些神经元的后续连接，从而使得这些神经元不再对外传递信号，不再对下一层神经元发生作用。训练时，根据训练结果更新未丢弃的神经元的参数，而丢弃的神经元的参数保持不变；下次训练时，重新随机选择需要丢弃（隐藏）的神经元。

BP算法过拟合问题的解决

丢弃方法如下图所示：标记了×号的神经元将会被从神经网络中丢弃，不再对下一层神经元发生作用。从而在一定程度上可以使得模型不再过度依赖某些神经元，达到抑制过拟合的效果。当然有得必有失，丢包也会造成信息的损失，使得模型的拟合相对不够充分，在实际应用时，主要目标是为了抑制过拟合时，使用该方法才是合适的。



BP算法过拟合问题的解决

三、正则化 (regularization)

正则化方法也成为权重衰减方法，其实现路径是：在神经网络的最优化函数中加入 L_2 惩罚项，或者说加入用于描述模型复杂度的部分，从而加大对过度训练、进而造成模型复杂程度增加的惩罚力度。

从数学公式的角度，即把前述最优化函数变为：

$$\underset{W}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n L[y_i, F(x_i; W)] + \lambda \|W\|_2^2$$

函数中的 $\|W\|_2^2$ 是神经元连接权重矩阵中所有元素的平方和，模型复杂程度越高， $\|W\|_2^2$ 的值就会越大，达到正则化的目的。 λ 为惩罚系数，可通过 K 折交叉验证法确定最优解。



PART 02

数据准备

数据准备

本章我们用到的数据为前面章节中介绍的“数据15.1”“数据13.1”“数据15.2”。

针对“数据15.1”，我们以pb为响应变量，以roe、debt、assetturnover、rdgrow、roic、rop、netincomeprofit、quickratio、incomegrow、netprofitgrow、cashflowgrow为特征变量，使用回归问题神经网络算法进行拟合。

针对“数据13.1”，我们以是否发生违约（credit）为响应变量，以年龄（age）、受教育程度（education）、工作年限（workyears）、居住年限（resideyears）、年收入水平（income）、债务收入比（debtratio）、信用卡负债（creditdebt）、其他负债（otherdebt）为特征变量，使用二分类神经网络算法进行拟合。

针对“数据15.2”，我们以grade为响应变量，以age、marry、years、income、education、consume、work、gender、family为特征变量，使用多分类神经网络算法进行拟合。

载入分析所需要的模块和函数

在进行分析之前，我们首先载入分析所需要的模块和函数，读取数据集并进行观察。

示例

参阅教材内容

PART 03

回归神经网络算法示例

|| 变量设置及数据处理

示例

参阅教材内容

|| 单隐藏层的多层感知机算法

示例

参阅教材内容

神经网络特征变量重要性水平分析

示例

参阅教材内容

|| 绘制部分依赖图与个体条件期望图

示例

参阅教材内容

|| 拟合优度随神经元个数变化的可视化展示

示例

参阅教材内容

|| 通过K折交叉验证寻求单隐藏层最优神经元个数

示例

参阅教材内容

|| 双隐藏层的多层感知机算法

示例

参阅教材内容

|| 最优模型拟合效果图形展示

示例

参阅教材内容

PART 04

二分类神经网络算法示例

|| 变量设置及数据处理

示例

参阅教材内容

|| 单隐藏层二分类问题神经网络算法

示例

参阅教材内容

|| 双隐藏层二分类问题神经网络算法

示例

参阅教材内容

|| 早停策略减少过拟合问题

示例

参阅教材内容

|| 正则化（权重衰减）策略减少过拟合问题

示例

参阅教材内容

模型性能评价

示例

参阅教材内容

|| 绘制ROC曲线

示例

参阅教材内容

|| 运用两个特征变量绘制二分类神经网络算法决策边界图

示例

参阅教材内容

PART 05

多分类神经网络算法示例

|| 变量设置及数据处理

示例

参阅教材内容

|| 单隐藏层多分类问题神经网络算法

示例

参阅教材内容

|| 双隐藏层多分类问题神经网络算法

示例

参阅教材内容

模型性能评价

示例

参阅教材内容

|| 运用两个特征变量绘制多分类神经网络算法决策边界图

示例

参阅教材内容



PART 06

习 题

针对“数据6.1”（在第6章中已详细介绍），我们以收入档次（V1）为响应变量，以工作年限（V2）、绩效考核得分（V3）和违规操作积分（V4）为特征变量，使用多分类神经网络算法进行拟合。

- (1) 变量设置及数据处理
- (2) 单隐藏层多分类问题神经网络算法
- (3) 双隐藏层多分类问题神经网络算法
- (4) 模型性能评价
- (5) 运用两个特征变量绘制多分类神经网络算法决策边界图